

RETAIL SALES FORECASTING SYSTEM

Using Machine Learning Models and Flask API

ABSTRACT

Retail sales forecasting is a crucial task for businesses to optimize inventory, improve decision-making, and reduce financial losses. This project develops a machine learning-based forecasting system that predicts future retail sales using multiple models. The system also includes a Flask-based REST API to serve predictions in real-time.

PROBLEM STATEMENT

Retail companies face challenges in accurately predicting future sales due to changing customer demand and seasonal variations. Poor forecasting leads to overstocking or stockouts, affecting profitability. The objective of this project is to build an intelligent forecasting system that predicts future sales using historical data and provides results through a web API.

DATASET DESCRIPTION

The dataset used in this project consists of historical retail sales data stored in Excel format.

Key Features:

- Weekly sales records
- Multiple states (e.g., Texas, California, Illinois)
- Time-series structured data

Purpose:

The dataset is used to analyze historical trends and train forecasting models to predict future sales values.

TECHNOLOGY STACK

- Python
- Pandas & NumPy
- Matplotlib & Seaborn
- Scikit-learn
- Statsmodels
- Facebook Prophet
- Flask (REST API)

METHODOLOGY

The project follows a structured machine learning pipeline:

1. Data Collection
2. Data Preprocessing (handling missing values, formatting time series)
3. Feature Engineering (lag features, rolling averages)
4. Model Training
5. Model Evaluation using RMSE
6. Best Model Selection
7. Future Forecast Generation (8 weeks)
8. Deployment using Flask API

MODELS USED

Multiple forecasting models were implemented to improve accuracy:

- SARIMA (Seasonal ARIMA)
- Facebook Prophet
- XGBoost Regressor
- LSTM (Deep Learning model)

Each model was evaluated, and the best-performing model was selected based on RMSE (Root Mean Squared Error).

SYSTEM ARCHITECTURE

The proposed system follows a modular machine learning pipeline designed for time-series forecasting. It begins with data collection from historical retail sales records, followed by preprocessing steps such as handling missing values, formatting time-series data, and ensuring consistency in the dataset. After preprocessing, feature engineering is applied to create meaningful input variables such as lag features and rolling statistics, which help improve model performance and capture temporal patterns in sales data.

Multiple forecasting models including SARIMA, Facebook Prophet, XGBoost, and LSTM are trained on the processed dataset. Each model is evaluated using performance metrics such as RMSE (Root Mean Squared Error) to measure prediction accuracy. Based on evaluation results, the best-performing model is selected for final forecasting. The selected model is then used to generate future sales predictions for the next 8 weeks. These predictions are stored as structured output files and visualized using plots for better interpretation.

Finally, the forecasting system is deployed using a Flask REST API, which allows users to request predictions for specific states. The API returns results in JSON format, making it easy to integrate with web or mobile applications. This architecture ensures modularity, scalability, and real-time accessibility of the forecasting system.

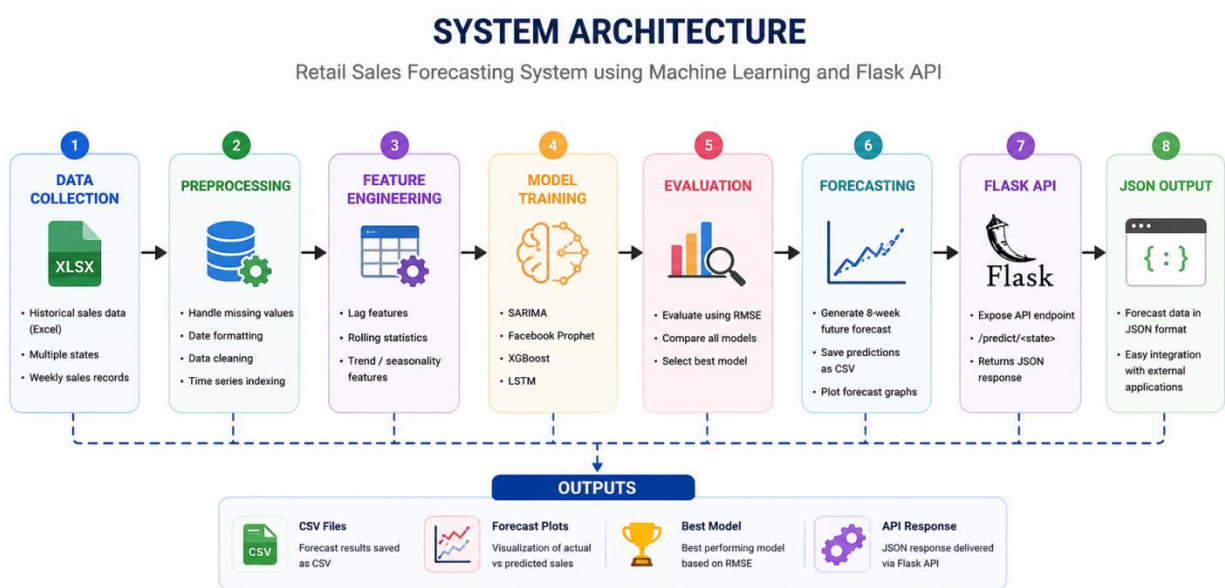


Figure 1: System Architecture of Retail Forecasting System

OUTPUTS GENERATED

The system produces the following outputs:

- 8-week future sales forecasts
- RMSE comparison between models
- Best model selection results
- CSV files containing predictions
- Forecast visualization plots

FLASK API IMPLEMENTATION

A REST API was developed using Flask to serve predictions dynamically.

API Endpoint:

`/predict/<state>`

Example:

`http://127.0.0.1:5000/predict/Texas`

Response Format:

```
{
  "state": "Texas",
  "forecast": [
    {
      "Week": 1,
      "Predicted_Sales": 936914300
    }
  ]
}
```

This enables real-time access to forecasted data.

RESULTS

- The system successfully generated accurate forecasts for multiple states.
- All models were evaluated using RMSE metrics.
- Flask API successfully returns JSON-based predictions.
- Forecast plots and CSV outputs were generated successfully.

Screenshots included:

- API terminal running

```
File Edit Selection View Go Run Terminal Help
microgoc_case_study

EXPLORER
  MICROGOC_CASE_STUDY
    api
    data
    sales_data.xlsx
    models
    notebooks
    outputs
    future_forecasts
    models
    plots
    predictions
    model_results.csv
    rmse_comparison.png
    src
    venv
    app.py
    main.py

PROBLEMS
DEBUG CONSOLE
TERMINAL
PORTS

41 Alabama LSTM 6.275441e+07 2.387649e+07 3.651845e+07 1.176624e+07
42 Missouri LSTM 5.548581e+07 1.706692e+07 3.963166e+07 1.600766e+07
(venv) PS C:\Users\VSP\Desktop\microgoc_case_study> pip install flask
Collecting flask
  Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Collecting werkzeug>=2.2.0
  Using cached werkzeug-3.1.3-py3-none-any.whl (226 kB)
Collecting itsdangerous>=2.2.0
  Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting markupsafe>=2.1.1
  Using cached markupsafe-3.0.3-cp310-cp310-win_amd64.whl (15 kB)
Requirement already satisfied: click>=8.1.3 in c:\users\vsp\desktop\microgoc_case_study\venv\lib\site-packages (from flask) (8.3.3)
Collecting blinker>=1.9.0
  Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting jinja2>=3.1.2
  Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Requirement already satisfied: colorama in c:\users\vsp\desktop\microgoc_case_study\venv\lib\site-packages (from click>=8.1.3->flask) (0.4.6)
Installing collected packages: markupsafe, itsdangerous, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 23.0.1 -> 26.1.1
[notice] To update, run 'python.exe -m pip install --upgrade pip'
(venv) PS C:\Users\VSP\Desktop\microgoc_case_study> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 236-013-627
127.0.0.1 - - [07/May/2026 20:48:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/May/2026 20:48:19] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [07/May/2026 20:52:07] "GET /predict/Texas HTTP/1.1" 200 -
127.0.0.1 - - [07/May/2026 20:58:12] "GET /predict/California HTTP/1.1" 200 -
127.0.0.1 - - [07/May/2026 20:58:27] "GET /predict/Illinois HTTP/1.1" 200 -
```

- JSON output in browser

```
Next Steps: Assessment x Case - Study - Forecast x 127.0.0.1:5000 x 127.0.0.1:5000/predict/ x 127.0.0.1:5000/predict/ x 127.0.0.1:5000/predict/ x + Ask Gemini x - x x
127.0.0.1:5000
Pretty-print
{
  "message": "Retail Forecasting API Running"
}
```



```
Next Steps: Assessment x Case - Study - Forecast x 127.0.0.1:5000 x 127.0.0.1:5000/predict/ x
127.0.0.1:5000/predict/Texas
pretty-print
{
  "forecast": [
    {
      "Predicted_Sales": 936914300.0,
      "Week": 1
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 2
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 3
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 4
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 5
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 6
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 7
    },
    {
      "Predicted_Sales": 936914300.0,
      "Week": 8
    }
  ],
  "state": "Texas"
}
```

- CSV forecast file

California_future.csv - Microsoft Excel

Home

Insert

Page Layout

Formulas

Data

Review

View

Paste

Clipboard

Font

Alignment

Number

General

\$

%

0.00

0.00

Number

Conditional Formatting

Format as Table

Cell Styles

Styles

Insert

Delete

Format

Cells

Σ

Sort & Filter

Find & Select

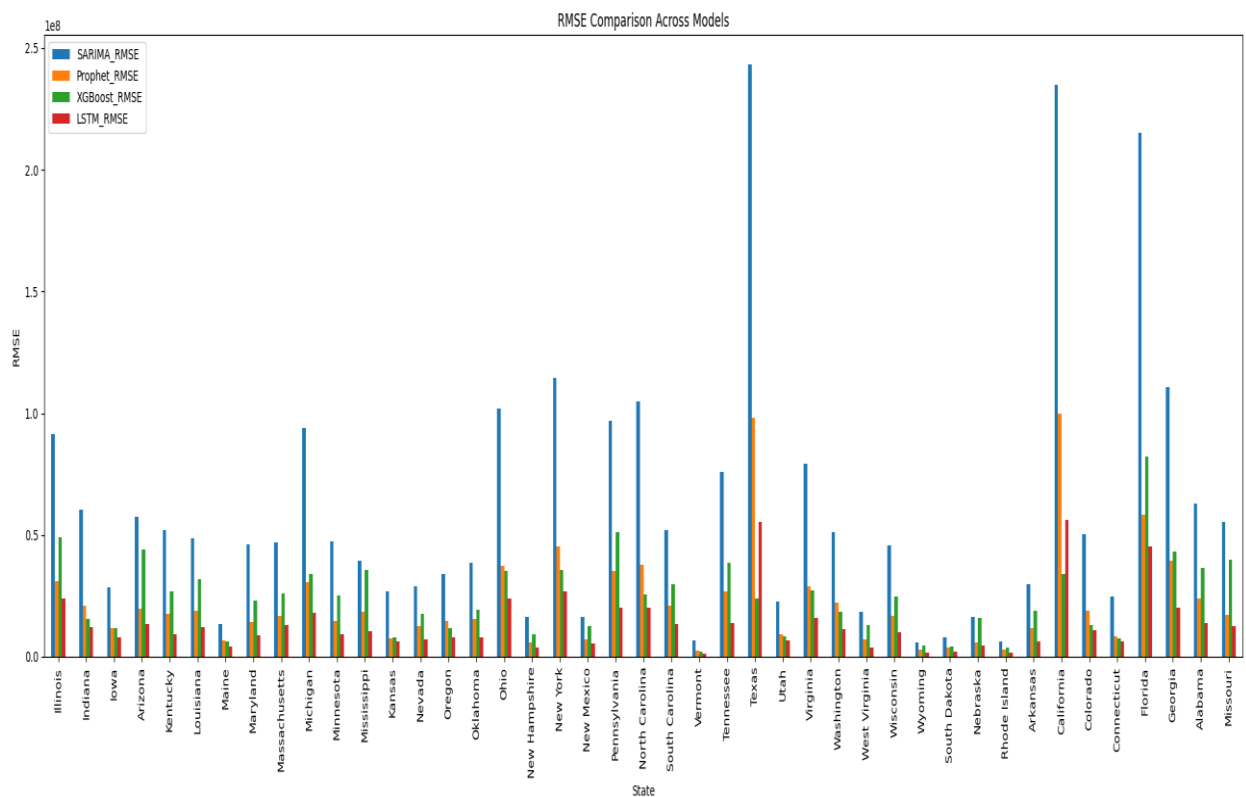
Editing

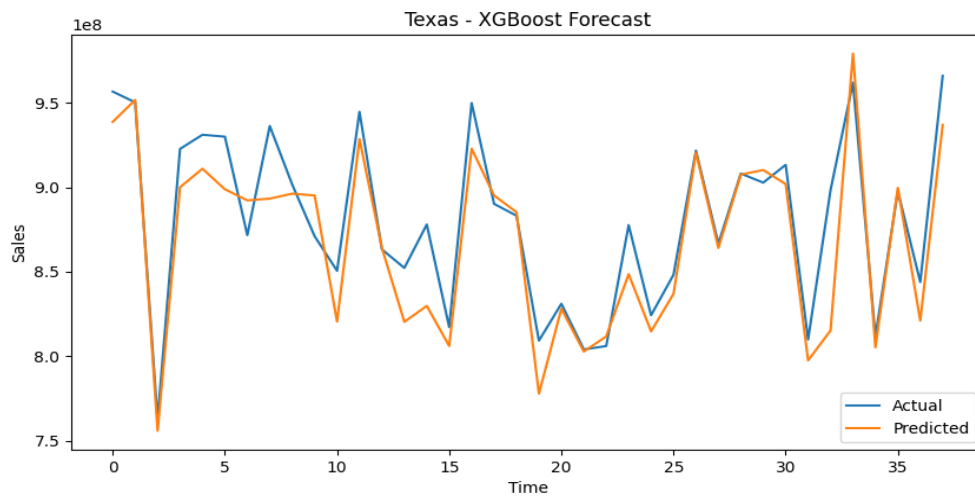
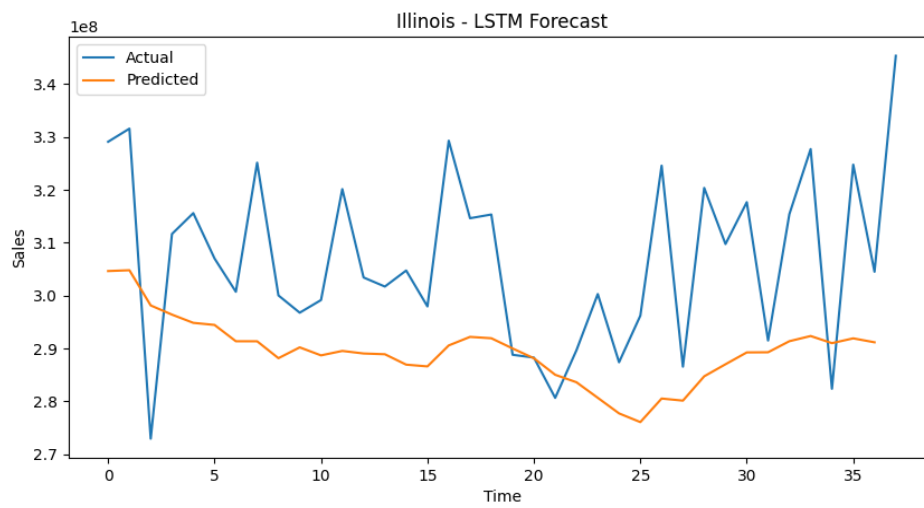
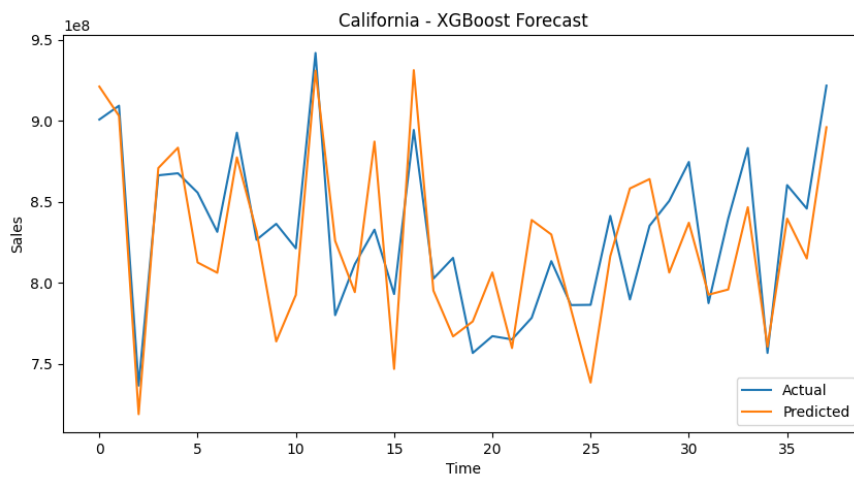
A1

Week

	A	B	C	D	E	F	G	H	I	J	K
1	Week	Predicted_Sales									
2	1	8.96E+08									
3	2	8.96E+08									
4	3	8.96E+08									
5	4	8.96E+08									
6	5	8.96E+08									
7	6	8.96E+08									
8	7	8.96E+08									
9	8	8.96E+08									
10											
11											

- Forecast visualization plots





CONCLUSION

This project successfully demonstrates a complete end-to-end machine learning pipeline for retail sales forecasting. It integrates multiple forecasting models and deploys the best model through a Flask API. The system is efficient, scalable, and suitable for real-world retail analytics applications.

FUTURE ENHANCEMENTS

- Integration with real-time streaming data
- Deployment on cloud platforms (AWS / Azure)
- Advanced deep learning models for improved accuracy
- Interactive dashboard for visualization